6. Source Code & Coding

This chapter showcases the core code components of the E-Commerce Multi-Vendor Platform. The system is split between a Python/Django REST Framework backend (organized into 5 apps: `users`, `vendors`, `products`, `orders`, `dashboard`) and a JavaScript/React frontend (organized into `pages`, `components`, `context`, and `services`). Six critical code components are documented below.

6.1 Django Custom User Model

Located in `backend/users/models.py`, this model overrides Django's default username-based authentication to use **email** as the primary identifier. It defines the three user roles (Customer, Vendor, Admin) that drive role-based access control (RBAC).

```
from django.contrib.auth.models import AbstractUser, BaseUserManager
from django.db import models

class UserManager(BaseUserManager):
    def create_user(self, email, username, password=None, role='customer'):
        if not email:
            raise ValueError('Email is required')
        email = self.normalize_email(email)
        user = self.model(email=email, username=username, role=role)
        user.set_password(password)
        user.save(using=self._db)
        return user

    def create_superuser(self, email, username, password=None):
        user = self.create_user(email, username, password, role='admin')
        user.is_staff = True
        user.is_superuser = True
        user.save(using=self._db)
        return user

class User(AbstractUser):
    ROLE_CHOICES = (
        ('customer', 'Customer'),
        ('vendor', 'Vendor'),
        ('admin', 'Admin'),
    )
    email    = models.EmailField(unique=True)
    role     = models.CharField(max_length=20, choices=ROLE_CHOICES, default='customer')
    phone    = models.CharField(max_length=15, blank=True)
    address  = models.TextField(blank=True)
    objects  = UserManager()

    USERNAME_FIELD  = 'email'
    REQUIRED_FIELDS = ['username']

    def __str__(self):
        return f"{self.email} ({self.role})"
```

6.2 Django Product ViewSet with RBAC Permissions

Located in `backend/products/views.py`, this viewset handles product CRUD operations and filters listings. It utilizes a custom permission class, `IsVendorOrReadOnly`, to enforce backend role-based access control.

```
from rest_framework import viewsets, permissions
from .models import Product, Category, ProductImage
from .serializers import ProductSerializer, CategorySerializer

class IsVendorOrReadOnly(permissions.BasePermission):
    def has_permission(self, request, view):
        if request.method in permissions.SAFE_METHODS:
            return True
        return request.user.is_authenticated and request.user.role == 'vendor'

    def has_object_permission(self, request, view, obj):
        if request.method in permissions.SAFE_METHODS:
            return True
        return obj.vendor.user == request.user

class ProductViewSet(viewsets.ModelViewSet):
    serializer_class   = ProductSerializer
    permission_classes = [IsVendorOrReadOnly]

    def get_queryset(self):
        qs       = Product.objects.filter(is_active=True).order_by('-created_at')
        search   = self.request.query_params.get('search')
        category = self.request.query_params.get('category')
        vendor   = self.request.query_params.get('vendor')

        if search:   qs = qs.filter(name__icontains=search)
        if category: qs = qs.filter(category__slug=category)
        if vendor:   qs = qs.filter(vendor__id=vendor)
        return qs

    def perform_create(self, serializer):
        product = serializer.save(vendor=self.request.user.vendor_profile)
        images = self.request.FILES.getlist('images')
        for image in images:
            ProductImage.objects.create(product=product, image=image)
```

6.3 React Axios Interceptor Service

Located in `frontend/src/services/api.js`, this service centralizes REST API requests. It automatically configures request headers with the active JWT access token, intercepts 401 Unauthorized responses to perform a silent refresh, and handles expired login sessions.

```
import axios from 'axios';

const api = axios.create({ baseURL: 'http://localhost:8000/api' });

api.interceptors.request.use(config => {
```

```
  const token = localStorage.getItem('access');
  if (token) config.headers.Authorization = `Bearer \${token}`;
  return config;
});

api.interceptors.response.use(
  res => res,
  async err => {
    if (err.response?.status === 401) {
      const refresh = localStorage.getItem('refresh');
      if (refresh) {
        try {
          const r = await axios.post('http://localhost:8000/api/auth/refresh/', { refresh
});
          localStorage.setItem('access', r.data.access);
          err.config.headers.Authorization = `Bearer \${r.data.access}`;
          return api(err.config);
        } catch {
          localStorage.removeItem('access');
          localStorage.removeItem('refresh');
          window.location.href = '/login';
        }
      }
    }
    return Promise.reject(err);
  }
);

export default api;
```

6.4 React Cart Context API

Located in `frontend/src/context/CartContext.jsx`, this Context provider maintains customer shopping cart state globally. It exposes handlers to add items, modify quantities, and clear items after checkout.

```
import { createContext, useContext, useState } from 'react';

const CartContext = createContext();

export function CartProvider({ children }) {
  const [cart, setCart] = useState([]);

  const addToCart = (product) => {
    setCart(prev => {
      const exists = prev.find(i => i.id === product.id);
      if (exists) return prev.map(i => i.id === product.id ? { ...i, qty: i.qty + 1 } : i);
      return [...prev, { ...product, qty: 1 }];
    });
  };

  const removeFromCart = (id) => setCart(prev => prev.filter(i => i.id !== id));

  const updateQty = (id, qty) => {
    if (qty < 1) return removeFromCart(id);
    setCart(prev => prev.map(i => i.id === id ? { ...i, qty } : i));
```

```
  };

  const clearCart = () => setCart([]);

  const total = cart.reduce((s, i) => s + parseFloat(i.price) * i.qty, 0);
  const count = cart.reduce((s, i) => s + i.qty, 0);

  return (
    <CartContext.Provider value={{ cart, addToCart, removeFromCart, updateQty, clearCart,
total, count }}>
      {children}
    </CartContext.Provider>
  );
}

export function useCart() { return useContext(CartContext); }
```

6.5 Django Order Checkout Serializer (with Stock Deduction)

Located in `backend/orders/serializers.py`, the `OrderCreateSerializer.create()` method is the heart of the checkout flow. It processes cart items sent from the React frontend, calculates the total, saves the Order header row, creates individual OrderItem rows with a frozen price snapshot, and decrements product stock — all in a single database operation.

```
class OrderCreateSerializer(serializers.Serializer):
    items   = OrderItemCreateSerializer(many=True)
    address = serializers.CharField()

    def create(self, validated_data):
        customer   = self.context['request'].user
        items_data = validated_data.pop('items')
        total      = 0
        item_objects = []

        for item in items_data:
            product  = Product.objects.get(id=item['product_id'])
            subtotal = product.price * item['quantity']
            total   += subtotal
            item_objects.append({
                'product':  product,
                'quantity': item['quantity'],
                'price':    product.price  # Freeze price at order time
            })

        order = Order.objects.create(
            customer=customer,
            total_amount=total,
            address=validated_data['address']
        )

        for i in item_objects:
            OrderItem.objects.create(order=order, **i)
            i['product'].stock -= i['quantity']  # Deduct stock
            i['product'].save()

        return order
```

6.6 React AuthContext Provider

Located in `frontend/src/context/AuthContext.jsx`, this Context provider persists authentication state across all components. On mount, it checks `localStorage` for a JWT token and silently restores session. The `login()` function stores tokens and fetches user profile in one flow; `logout()` clears tokens and wipes state.

```
export function AuthProvider({ children }) {
  const [user, setUser]       = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const token = localStorage.getItem('access');
    if (token) {
      api.get('/auth/me/')
        .then(res => setUser(res.data))
        .catch(() => {
          localStorage.removeItem('access');
          localStorage.removeItem('refresh');
        })
        .finally(() => setLoading(false));
    } else { setLoading(false); }
  }, []);

  const login = async (email, password) => {
    const res = await api.post('/auth/login/', { email, password });
    localStorage.setItem('access',  res.data.access);
    localStorage.setItem('refresh', res.data.refresh);
    const me = await api.get('/auth/me/');
    setUser(me.data);
    return me.data;
  };

  const logout = () => {
    localStorage.removeItem('access');
    localStorage.removeItem('refresh');
    setUser(null);
  };

  return (
    <AuthContext.Provider value={{ user, login, logout, loading, setUser }}>
      {children}
    </AuthContext.Provider>
  );
}
```

6.7 Code Integration Flow Diagram

The following sequence diagram details how the six core code components integrate to process a client request — starting from user authentication to verifying custom backend permissions and executing order checkout.

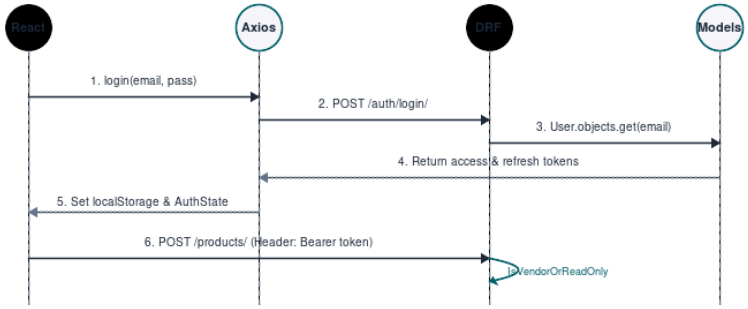

Figure 1 — Sequence flow of token-based requests authentication and authorization